


# OPERATING SYSTEMS

## UE25CS242A

## Memory Management

**Pavan A C and Nivedita Kasturi**

Department of Computer Science & Engineering

# OPERATING SYSTEMS

## Slides Credits for all the PPTs of this course




- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

# OPERATING SYSTEMS

PES UNIVERSITY
CELEBRATING 50 YEARS

## Main Memory:

## Hardware and control structures, OS support, Address translation

**Pavan A C and Nivedita Kasturi**
Department of Computer Science & Engineering

## Background

- What is a memory?
  - Memory consists of a large array of bytes, each with its own address.
- Execution of an instruction,
  - Fetch an Instruction from memory, Decode the instruction, operands are fetched(from memory or registers)
  - After the instruction is executed, results are stored back.
- The memory unit(MU) sees the stream of addresses.
- Memory unit does not know how these addresses are generated.
- We will learn how the addresses are generated by the running program.

PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Basic Hardware




- Program must be brought (from disk) into memory and placed within a ready queue it to be run
- Main memory and registers are only storage CPU can access directly
- The data required for CPU must be made available in the registers.
  - Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
  - Speeds up memory access without any OS control
- Protection of memory required to ensure correct operation

# OPERATING SYSTEMS

## Multi-Programming

- Protection of OS from its access by user processes is needed
- On multiuser systems user processes must be protected from each other.
- This protection must be provided by the hardware because the operating system doesn’t usually intervene between the CPU and its memory accesses.
- Several hardware protection methods will be discussed.
- Protection by using two registers, usually a base and a limit.

PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Multi-Programming

- Protection of OS from its access by user processes is needed
- On multiuser systems user processes must be protected from each other.
- This protection must be provided by the hardware because the operating system doesn’t usually intervene between the CPU and its memory accesses.
- Several hardware protection methods will be discussed.
- Protection by using two registers, usually a base and a limit.

PES UNIVERSITY
CELEBRATING 50 YEARS

## Protection vs. Translation

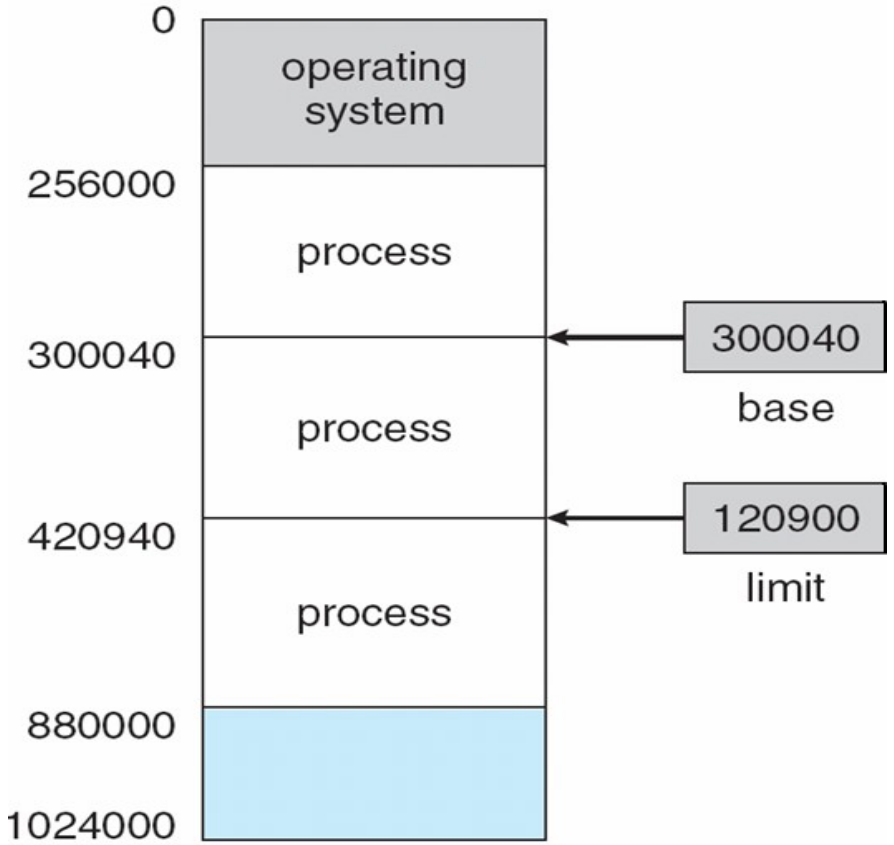
- Protection - preventing access to private memory of other processes
- Translation - converting virtual addresses to physical ones to provide a uniform memory view

PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Basic Hardware

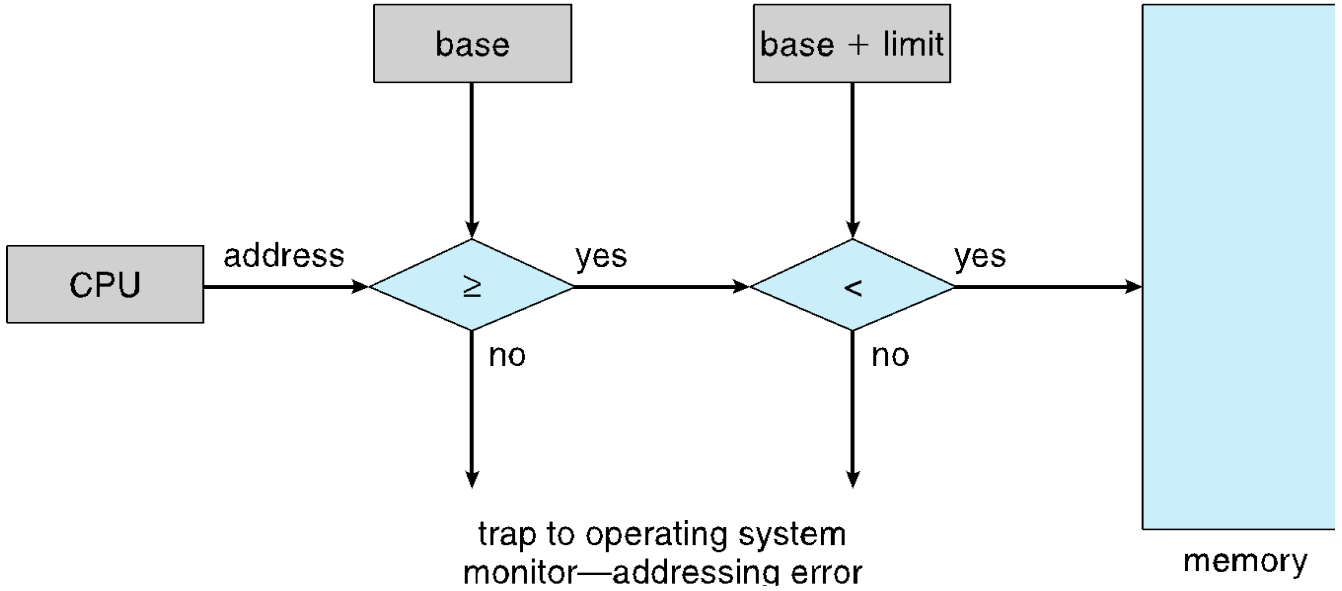
- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



# OPERATING SYSTEMS

## Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user




- prevents a user program from modifying the code or data structures of either the operating system or other users.

## Basic Hardware

- The base and limit registers can be loaded only by the operating system.
  - Using special privileged instruction.
  - privileged instructions can be executed only in kernel mode.
  - OS runs in the kernel mode. Only OS can change the register values.
  - This prevents other user programs to modify the register values.

PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Address Binding




- Programs on disk as binary executable and are brought to main memory for execution
- Processes may be moved between the disk and memory during execution.
- What are the steps involved in the program execution?
- Most systems allow a user process to reside in any part of the physical memory.
- The address space of the computer may start at 00000 but the first address of the user process need not be 00000
- Addresses in the source program are generally symbolic (Ex. variable count).

## Address Binding




- A compiler typically binds these symbolic addresses to relocatable addresses
  - “14 bytes from the beginning of this module”
- The linkage editor or loader in turn binds the relocatable addresses to absolute addresses
  - Ex. 74014
- Each binding is a mapping from one address space to another.

## Memory-Management Unit (Cont.)

Base-register scheme for address mapping

- The base register also referred to as a relocation register.
- The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory .
- Example, if the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000;
- An access to location 346 is mapped to location 14346.
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses

PES UNIVERSITY
CELEBRATING 50 YEARS

## Memory-Management Unit (Cont.)

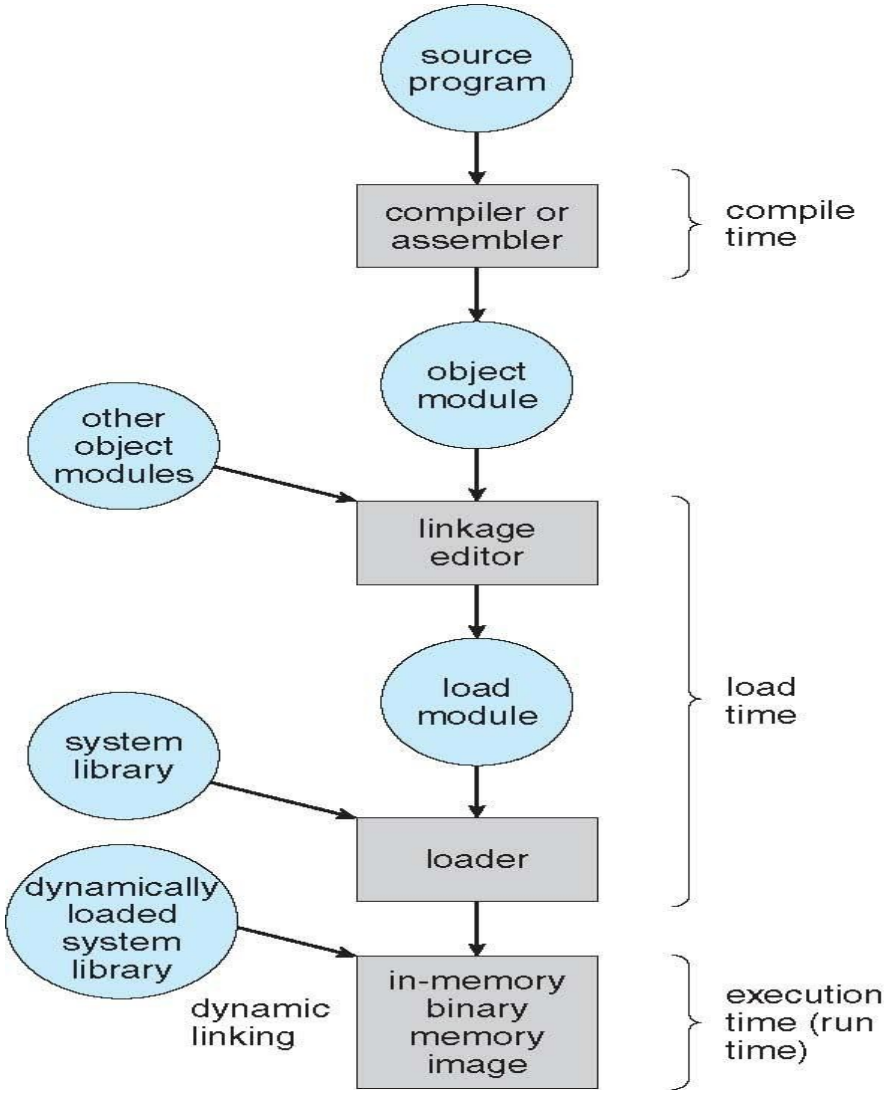
The binding of instructions and data to memory addresses

- **Compile time**. If you know at compile time where the process will reside in memory, then **absolute code** can be generated.
  - If the start address changes, it is necessary to recompile once again
  - The MS-DOS .COM-format programs are bound at compile time.
- **Load time**. If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code**.
  - final binding is delayed until load time.
  - If the starting address changes, we need only reload the user code to incorporate this changed value.
- **Execution time**. If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.
  - Special hardware must be available for this scheme to work

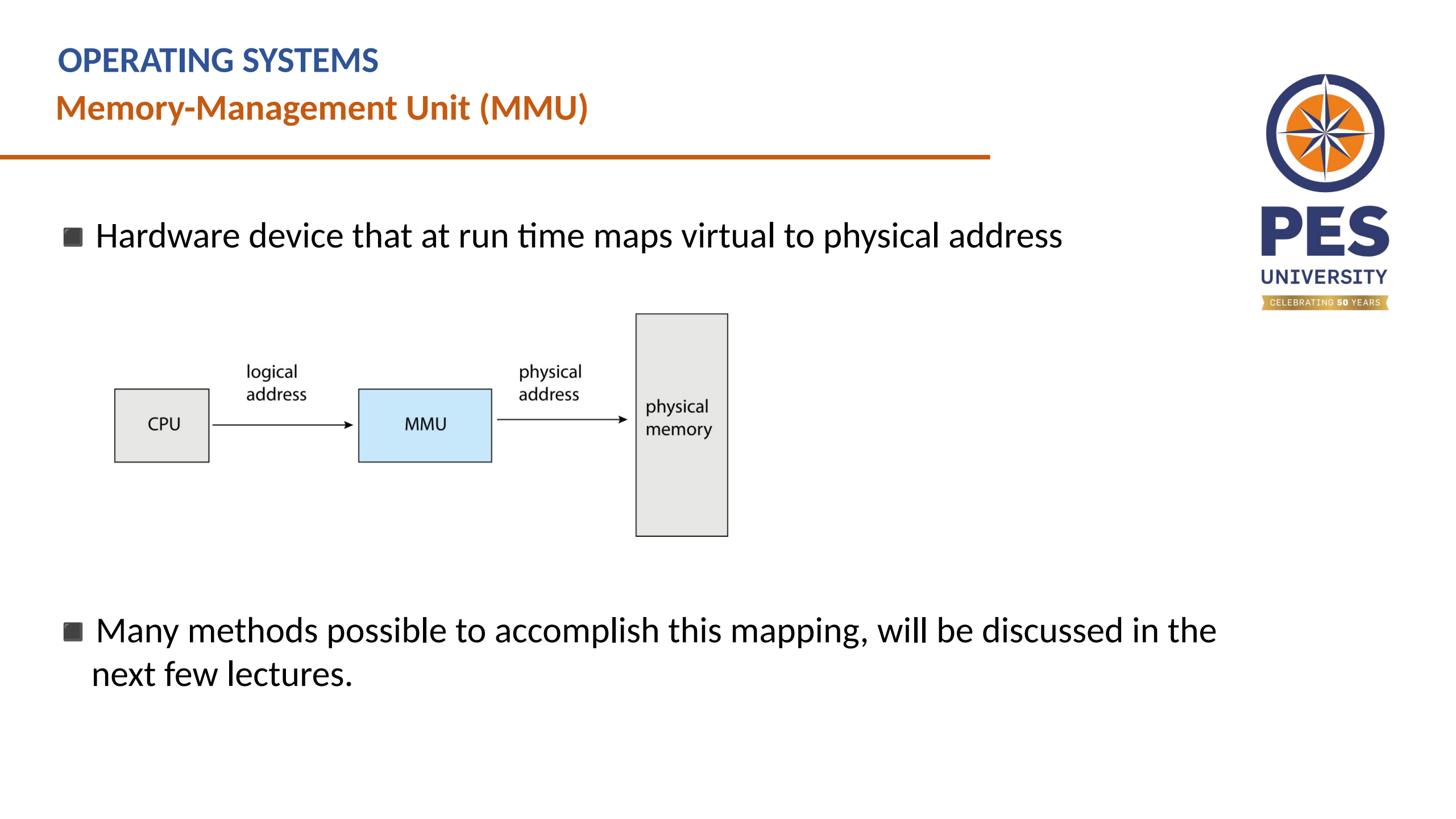
PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Multistep Processing of a User Program



# Memory-Management Unit (MMU)




- Hardware device that at run time maps virtual to physical address




- Many methods possible to accomplish this mapping, will be discussed in the next few lectures.

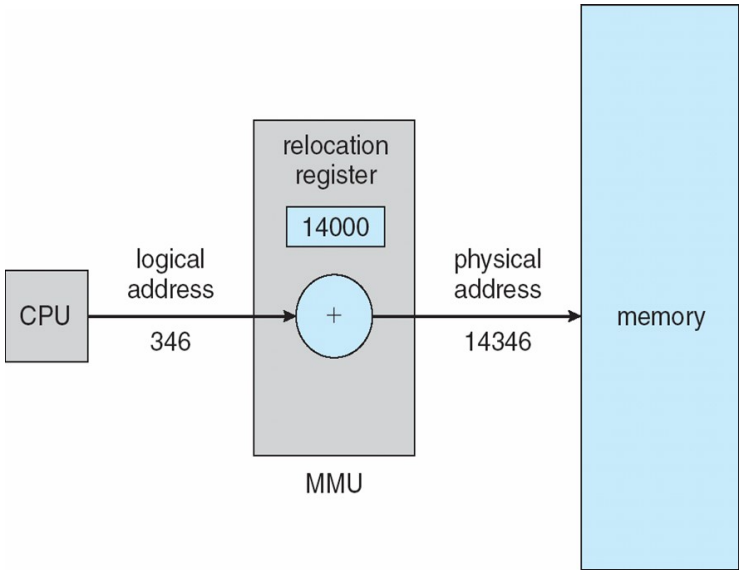
# Logical vs. Physical Address Space




- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

# Dynamic relocation using a relocation register

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading




PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Dynamic Linking




- **Static linking** – system libraries and program code combined by the loader into the binary program image
- **Dynamic linking** –linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
  - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- Other programs linked before the new library was installed will continue using the older library
- This System also known as **shared libraries**
- Consider applicability to patching system libraries
  - Versioning may be needed

## Static and Dynamic Linking

- A program whose necessary library functions are embedded directly in the program's executable binary file is ***statically*** linked to its libraries
- The main disadvantage of static linkage is that every program generated must contain copies of exactly the same common system library functions
- *Dynamic* linking is more efficient in terms of both physical memory and disk-space usage because it loads the system libraries into memory only once

PES UNIVERSITY
CELEBRATING 50 YEARS

# OPERATING SYSTEMS

## Memory-Mapped Files (mmap)

Mapping file contents directly into the process virtual address space.Access disk files via memory pointers (ptr[i]) instead of standard I/O system calls.Eliminate data duplication between kernel buffers and user space.

**How It Works:**

Address Translation: OS allocates a virtual address range to the file without loading it into physical RAM.

OS Support & Page Faults: Reading an address triggers a hardware page fault; the OS then pages the disk content into Main Memory.

Dynamic Shared Libraries: Uses the same mechanism as shared libraries to allow multiple processes to share memory safely.

**Two Core Modes**

Shared Mapping (MAP_SHARED): Writes modify the actual file on disk and are instantly visible to other processes.

Private Mapping (MAP_PRIVATE): Utilizes Copy-on-Write (CoW); modifications stay in memory and never change the disk file.

**Advantages**

High Performance: Bypasses overhead from read() and write() system calls.

Zero-Copy I/O: Transfers data directly from disk cache to user space.

Efficient IPC: Enables lightning-fast Inter-Process Communication via shared physical pages.

# Memory-Mapped Files- Life cycle of the Mapped Pages

1. First access:A demand-paging fault occurs; a page-sized chunk of the file is read from disk into a physical page.
2. Subsequent access:Reads and writes to the file are handled as ordinary memory accesses — no read()/write() system call overhead.
3. Deferred write-back:Modified pages aren't written to disk immediately; under memory pressure they're paged to swap space instead of being lost.
4. On close:All memory-mapped data is written back to secondary storage and unmapped from the process's virtual memory.

**Solaris Example:**

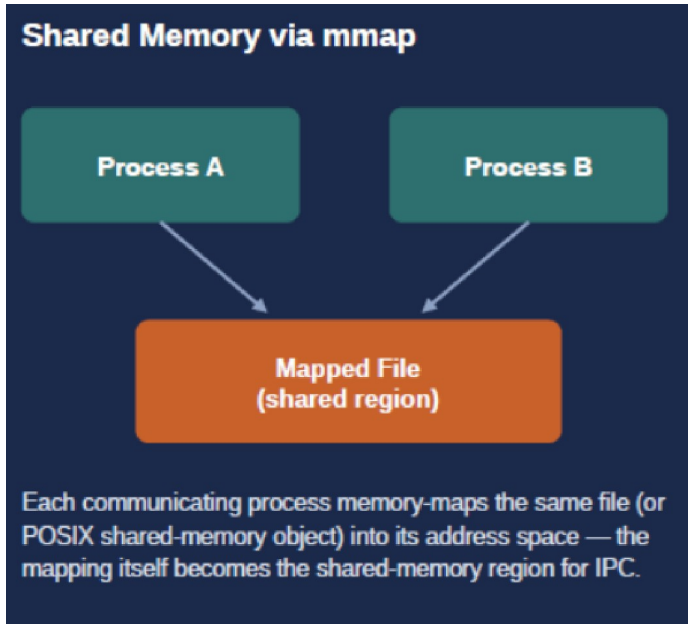
- mmap(): File is mapped directly into the process's address space.
- open() / read() / write(): File is still memory-mapped — but into kernel address space.
- Net effect: All file I/O goes through the efficient memory subsystem, avoiding traditional system-call overhead.

# Memory-Mapped Files: Advantages and Sharing Memory

- **Faster file access**
  - The file can be accessed directly through memory addresses.
  - This can be faster than repeatedly using read() and write() system calls.
- **Less data copying**
  - The operating system can map file pages directly into the process's address space.
  - This reduces unnecessary copying of data between kernel buffers and user-space buffers.
- **Random access is easy**
  - Any part of the file can be accessed directly using an address.
  - There is no need to sequentially read the file to reach a particular location.
- **Efficient for large files**
  - The entire file does not necessarily need to be loaded into RAM.
  - The OS loads required pages into memory when they are accessed using **demand paging**.
- **Easy programming**
  - After mapping, the file can be accessed using normal pointer/array operations.
  - For example:
  - data[100]
  - can access data at a particular location in the mapped file.
- **Sharing between processes**
  - Multiple processes can map the same file and potentially share the same physical memory pages.
  - This can be useful for **IPC and shared data**.
- **Changes can be reflected in the file**
  - With a shared mapping (MAP_SHARED), modifications to the mapped memory can be written back to the underlying file.

# Memory-Mapped Files: Advantages and Sharing Memory

- **Concurrent access, copy-on-write, and shared memory**
- **Concurrent Sharing**
  Multiple processes can map the same file at once. Each process's virtual memory map points to the same physical page — the page holding the disk block — so writes by one process are immediately visible to all others.
- **Copy-on-Write**
  Mapping calls can support copy-on-write: processes share a file read-only, but any process that modifies data gets its own private copy of just that data.




- Coordinating Access
  Since the memory is shared, processes typically use a mutual-exclusion mechanism to coordinate access to the shared data.
- Shared Memory via mmap (diagram box)
- Process A → Mapped File (shared region)
- Process B → Mapped File (shared region)
- Each communicating process memory-maps the same file (or POSIX shared-memory object) into its address space — the mapping itself becomes the shared-memory region for IPC.

## Memory-Mapped Files: Example

# mmap() Walkthrough — Reading a File

*Compare this to the traditional read() approach as we run it*

```c
// mmap_read.c
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    int fd = open("data.txt", O_RDONLY);
    struct stat sb;
    fstat(fd, &sb);                    // get file size
    char *map = mmap(NULL, sb.st_size, PROT_READ,
                     MAP_PRIVATE, fd, 0);
    for (int i = 0; i < sb.st_size; i++)
        putchar(map[i]);               // read via pointer
    munmap(map, sb.st_size); close(fd); return 0;
}
```

```
Run it:  gcc mmap_read.c -o mmap_read  &&  ./mmap_read
```

**open()**

OS creates a file descriptor; no data is read yet.

**mmap()**

OS reserves a virtual address range and builds page-table entries — file is still not in physical RAM.

**map[0]  (first touch**

CPU can't resolve the address → hardware page fault → OS reads that page from disk into a physical frame.

**map[1] … map[n]**

Same page already resident → handled as an ordinary memory access, no fault, no syscall.

**munmap() / close()**

Mapping is torn down; any modified pages are flushed back to disk.

## Memory-Mapped Files: Example

# MAP_SHARED — Two Processes, One File

```
// mmap_write.c  — run this in one terminal
int fd = open("data.txt", O_RDWR);
char *map = mmap(NULL, len, PROT_READ | PROT_WRITE,
                 MAP_SHARED, fd, 0);

map[0] = 'Z';          // modify in place, no write() call
// change is written straight through to the file
```

TRY THIS

- Open data.txt in a text editor / cat it in a second terminal before and after running mmap_write.
- Because the mapping is MAP_SHARED, the change is visible to any other process mapping the same file — no message-passing needed.
- Re-run with MAP_PRIVATE instead: the file on disk stays untouched — that's copy-on-write.

For Complete Example refer to **Lecture_30_Write Program _mmap_shared_guide.docx**

**Lecture_30_mmap_ reader_and_ipc_gui de.docx**

## Questions

- Q1: Why must memory protection checks (like base and limit) be implemented in hardware rather than by the OS software?
- Q2: What happens if a user process attempts to access a memory address that is greater than its limit register?
- Q3: Can a user process modify its own base and limit registers to access more memory? Why or why not?
- Q5: What is the primary disadvantage of static linking?
- Q6: What is a "stub" in the context of dynamic linking?
- Q7: Explain the concept of "Execution-time address binding."
- Q8: If a system uses a relocation (base) register set to 14000, what physical address is accessed if the program requests logical address 346?

PES UNIVERSITY
CELEBRATING 50 YEARS

## Questions

- Question 1: Explain the difference between MAP_SHARED and MAP_PRIVATE flags when used with the mmap() system call. What happens to modifications made on disk in each case?
- Question 2: In the code segment map[0] = 'Z';, how does the operating system handle writing this modification back to storage without a standard write() system call?
- Question 3: Why is it necessary to open the file with O_RDWR if you plan to use PROT_WRITE inside mmap()?
- Question 4: Suppose two independent processes use MAP_SHARED to map the exact same file region. If Process A changes map[0] to 'Z', how does Process B find out about the change? Does it require an explicit message-passing mechanism? Explain your answer.
- Question 5: What would happen if you ran the mmap_write program shown on the slide, but the file data.txt did not exist or was completely empty (0 bytes) before execution?




# THANK YOU

**Pavan A C**
Department of Computer Science and Engineering

**pavanac@pes.edu**